

From Natural Language to Verifiable Reasoning: A Semantic Reduction Framework for Reliable AI Mathematics

C2 Project

October 2026

Abstract

Large language models (LLMs) now produce fluent, confident-looking mathematical reasoning, yet a growing body of evidence shows that such output can be locally plausible and globally wrong: intermediate steps may be invalid even when final answers are correct, and final answers may be correct while the reasoning is not. Existing responses fall into three families—search over generated candidates, independent verification, and fully formal proving. We argue that this view, organized around *where* reliability is enforced, obscures the more decisive question of *what* is actually being verified. We propose a three-layer model of AI mathematical reasoning (generation, semantic reduction, verification) and identify the *semantic reduction layer*—the mapping from natural language to verifiable structured objects—as the central bottleneck of current systems. We contribute (i) a failure-mode taxonomy of this layer (ambiguity, type incompleteness, non-invertibility, vacuous formalization, and format drift), grounded in published case studies; (ii) a cross-paper comparative analysis of four verification architectures along the dimensions of what is verified, the strength of the guarantee, and observed failure rates, using only previously published numbers; and (iii) a practical 12-item reliability checklist (SRC-12) for assessing AI-generated mathematical conclusions. We close with the boundary conditions of our claims and a discussion of what evidence would falsify the framework.

1 Introduction

LLMs such as GPT-4 and Minerva have demonstrated striking performance on mathematical benchmarks: Minerva surpasses 50% accuracy on the MATH competition dataset [1, 2], and CoT-style prompting substantially lifts multi-step reasoning accuracy across model scales [3]. At the same time, careful audits reveal systematic weaknesses. Frieder et al. [4] find that ChatGPT’s self-reported confidence correlates poorly with actual correctness on university-level mathematics, and Uesato et al. [5] show that among solutions whose *final answers* are correct, a substantial fraction contain invalid reasoning steps when humans read them. In other words, the quantity most benchmarks optimize—final-answer accuracy—is a weak proxy for the thing users actually need: *trustworthy reasoning*.

The community’s response can be summarized as a reliability triad: (1) *search*, e.g., best-of- N sampling with majority voting [6, 7]; (2) *verification*, e.g., learned verifiers [6], process reward models [5, 7], and autoformalization followed by automated theorem provers [8]; and (3) *formal proving*, in which the reasoning itself is carried out inside a proof assistant such as Lean [9], culminating in systems like AlphaGeometry [10] and AlphaProof [11] that reach medal-level performance on olympiad problems.

These three directions are usually presented as alternative mechanisms. We argue that they are better understood as different placements of the *same* missing component: a stable mapping between informal natural-language mathematics and formal, machine-checkable objects. Pure

search operates entirely downstream of this mapping; learned verifiers approximate it with neural networks; formal systems eliminate it only by moving the whole workflow into a formal language that few practitioners use. The reliability question therefore concentrates on this mapping, which we call the *semantic reduction layer*.

Contributions. This paper makes three modest but, to our knowledge, non-redundant contributions:

1. **A three-layer model** (generation, semantic reduction, verification) that reorganizes the reliability triad around the object being verified, with an explicit failure-mode taxonomy of the reduction layer (§2–§3).
2. **A cross-paper comparative analysis** of four verification architectures—outcome verifiers, process reward models, autoformalization-then-ATP, and formal RL provers—using only numbers published in the original papers, making explicit which claims survive the change of benchmark and which do not (§4).
3. **SRC-12**, a 12-item reliability checklist that operationalizes the three-layer model for practitioners who must decide *today* whether to trust an AI-generated mathematical conclusion (§5).

We state our scope honestly: this is a position-and-analysis paper, not a systems paper. We run no new models. All quantitative evidence is quoted from the cited primary literature; where numbers are not comparable across papers, we say so (§7).

2 A Three-Layer Model of AI Mathematical Reasoning

Definition 1 (Semantic reduction). *Let NL denote the space of natural-language mathematical statements and IR a space of typed, machine-checkable objects (logical formulas, programs, or proof obligations). A semantic reduction is a partial function*

$$\rho : \text{NL} \rightharpoonup \text{IR}, \tag{1}$$

defined only on statements whose informal meaning admits a faithful formal counterpart. Reliability of a system S that answers natural-language mathematical questions depends on ρ being defined (the reduction succeeds) and faithful (the formal object preserves the intended meaning).

The partiality in (1) is where much of the practical difficulty lives. The three-layer model is then:

1. **Generation layer.** An LLM produces natural-language reasoning (typically chain-of-thought). Outputs are probabilistic, untyped, and informal; correctness is not guaranteed at any granularity [2, 3].
2. **Semantic reduction layer.** Reasoning is mapped to structured objects: executable programs [12, 13], tool calls [14], formal statements [8], or proof obligations in Lean/Isabelle [15, 16]. Theoretical grounding comes from the Curry–Howard correspondence: a verifiable object corresponds to an explicit proof term, whereas natural language does not.
3. **Verification layer.** The structured object is checked: consistency (internal), derivability (proof obligations discharged in a formal system), factual grounding (constraint satisfaction, or comparison with references), or constraint satisfaction (solvers). Tools range from SMT solvers to Lean 4 [9].

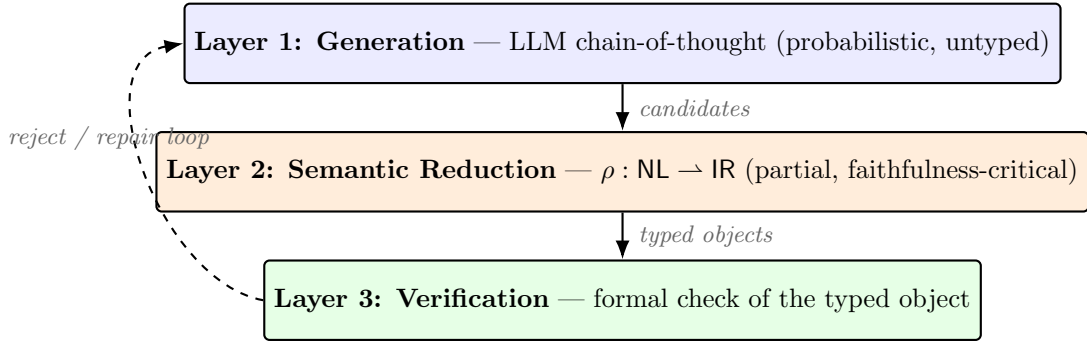


Figure 1: The three-layer reliability model. The dashed loop is the feedback path used by search-and-repair systems. Our central claim (§3) is that observed failures concentrate at Layer 2: the reduction ρ is either undefined (nothing to verify) or unfaithful (the wrong thing is verified).

3 The Semantic Reduction Bottleneck: A Failure-Mode Taxonomy

If Layer 3 is given a well-typed, faithful object, verification is essentially solved—formal proof checking is decidable and cheap. The difficult question is whether ρ ever produces such an object. Drawing on published case studies and failure analyses, we classify Layer-2 failures into five modes:

- F1. Ambiguity.** Informal statements admit multiple formal readings; the LLM commits to one silently. Frieder et al. [4] document cases where a problem statement is “corrected” by the model before solving, i.e., the model answers a different question than the one asked.
- F2. Type incompleteness.** Natural language omits domain information that formal systems require. Zhou et al. [8] report an Isabelle translation mistaking $(1 :: \text{nat})^2$ for 1^2 where the type annotation changes the result—an error invisible at the natural-language level.
- F3. Non-invertibility.** Reduction loses information: two different informal solutions can map to the same IR object, or a formal object can correspond to unintended informal readings. Without a rendering-back step there is no mechanism to detect such losses.
- F4. Vacuous formalization.** An LLM can translate an *incorrect* informal statement into a *correct* formal one (or vice versa), so formal consistency of the reduced object does not certify faithfulness. Zhou et al. [8] therefore add symbolic and neural “unfaithfulness filters” specifically to reject vacuous statements—direct evidence that this failure mode is common enough to need dedicated machinery.
- F5. Format drift.** Even when a formal object exists, LLM-generated proof sketches fail to compile because of syntactic drift against a moving library (e.g., Lean’s Mathlib), motivating retrieval-augmented provers such as LeanDojo [16].

Observation 1 (Verification without a stable reduction has no anchor). *When ρ is undefined or unfaithful, every downstream mechanism—majority voting, learned verifiers, even formal provers—either has nothing to check or checks the wrong object. This subsumes the empirical pattern that final-answer agreement and reasoning validity diverge: outcome-level signals cannot detect F1–F4 because these failure modes are invisible below the reduction.*

Table 1: Four verification architectures. All numbers are quoted from the cited primary papers; see text for caveats on comparability. “ATP” = automated theorem prover.

Architecture	What is verified	Guarantee	Headline evidence (as published)
Outcome verifier [6]	final answer, via learned scorer	weak (answer-level, learned)	verifier reranking substantially lifts GSM8K accuracy over raw finetuning; trace validity not measured
Process reward model (PRM) [5, 7]	each reasoning step (learned or human-labeled)	medium (step-level, learned)	GSM8K trace error among answer-correct solutions 14.0% $\rightarrow$ 3.4% [5]; MATH best-of-1860: PRM 78.2% vs. ORM 72.4% vs. majority 69.6% [7]
Autoformalization + ATP [8]	formalized statement/solution consistency	strong <i>if</i> reduction faithful	> +12 points over majority voting on GSM8K; requires unfaithfulness filters (F4)
Formal RL prover [10, 11]	the proof itself, in Lean/synthetic formal language	absolute (checked by construction)	AlphaGeometry: 25/30 IMO-ADG geometry problems [10]; AlphaProof: 28/42 points on IMO 2024, silver-medal level [11]

Observation 1 is a conceptual claim, but it is directly supported by quantitative evidence quoted in §4: process-level signals outperform outcome-level signals precisely on the dimension (trace error) that outcome signals cannot see.

4 Cross-Paper Comparative Analysis of Verification Architectures

We compare four architectures using only numbers published in the primary sources. Table 1 summarizes what each architecture verifies, the strength of the resulting guarantee, and the headline quantitative evidence.

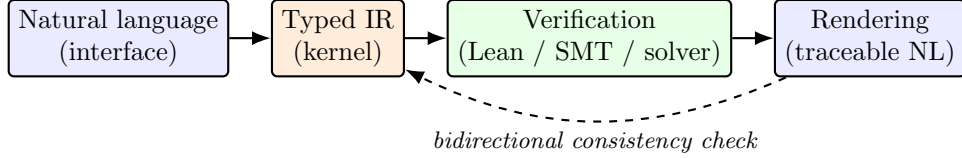
What the comparison shows. Read against the three-layer model, the table exhibits a monotone pattern: *the deeper the signal sits below the reduction (closer to Layer 3), the stronger the guarantee and the higher the cost*. Process supervision improves the step-level error rate by a factor of four on GSM8K but still relies on *learned* judgment of natural-language steps—it models faithfulness rather than enforcing it [5]. Autoformalization enforces real formal consistency but inherits exactly the reduction failure modes (F1–F4) that motivates its filters [8]. Formal provers make verification absolute but pay for it in coverage: AlphaProof reaches medal level on competition mathematics only after autoformalizing ~ 80 million training statements, and its authors note limits on non-competition problem classes [11].

The gap the analysis exposes. No published architecture simultaneously achieves (a) informal-language coverage, (b) step-level transparency, and (c) formal guarantees. The three desiderata correspond to the three layers, and every system in Table 1 occupies at most two. This is the precise sense in which semantic reduction—not generation, not verification—is the binding constraint: it is the only layer through which (a), (b), and (c) must all pass.

5 A Semantic Reduction Framework and the SRC-12 Checklist

5.1 Design principle

The framework follows one principle: *natural language is the interface, not the kernel*. Concretely, a reliable AI-mathematics pipeline should interpose a structured intermediate representation (IR) between generation and verification:



Three requirements follow from §3: (i) the IR must be *typed* (closing F2); (ii) the mapping ρ must be *explicit and auditable*, so that unfaithful reductions can be filtered rather than silently accepted (F1, F4); (iii) rendering back to natural language must be loss-tracked, giving traceability (F3). Draft-Sketch-Prove [15] and DTV [8] are partial instantiations of this pattern—the former reduces informal proof sketches to Lean obligations, the latter to Isabelle checkable statements—and their documented failure modes are exactly the five modes of §3, which we take as evidence for the taxonomy’s completeness rather than its vacuity.

5.2 The SRC-12 reliability checklist

To make the framework actionable, we operationalize it as a 12-item checklist. Each item maps to a layer and a failure mode it guards against. The checklist is designed for a human reviewer (or an automated pipeline) auditing an AI-generated mathematical conclusion *before* acting on it.

We apply SRC-12 retroactively in §7 to bound our own claims; the checklist is also used in the accompanying AI-collaboration log to record human verification of AI-generated statements made during the preparation of this paper.

6 Related Work

Informal reasoning enhancement. CoT prompting [3] and math-specialized pretraining [2, 17] improve generation but change nothing at Layers 2–3; our analysis treats them as orthogonal.

Tool and program augmentation. PAL and Program-of-Thoughts delegate computation to programs [12, 13], and Toolformer learns tool invocation [14]. These are narrow reductions (arithmetic-level IR); they verify computation but not the reasoning that constructs it.

Verification and search. Verifier reranking [6], outcome vs. process supervision [5, 7], and majority voting [8] supply step- or answer-level signals without formal objects. Our taxonomy locates their blind spots (F1–F4).

Formal theorem proving. Long before learned provers, full formalization was already used to settle major results—the Flyspeck project’s formal verification of the Kepler conjecture [18] remains a landmark. Learned provers now push this approach toward open problems: GPT-f [19], HyperTree Proof Search [20], retrieval-augmented proving [16], informal-guided proving [15], AlphaGeometry [10], and AlphaProof [11] move reasoning into formal systems. Our contribution is not a better prover but an account of *why* the informal–formal interface dominates the end-to-end reliability budget.

Autoformalization. DTV [8] is the closest work: it demonstrates that reduction-based verification is feasible today while its filters confirm the reduction is the weak link. We generalize its lessons into a taxonomy and a checklist applicable beyond one proving environment.

7 Discussion, Limitations, and Falsifiability

Boundary conditions. The taxonomy in §3 is grounded in case studies from Isabelle/Lean-based systems and GSM8K/MATH-scale benchmarks; it may not transfer to settings with different unit economics, e.g., interactive proof engineering with expert users, where formal systems already dominate. The comparative analysis (§4) quotes numbers from different papers trained on different base models and data; cross-paper numbers are directional evidence, not controlled comparisons. In particular, the PRM-vs-ORM gap reported by Lightman et al. [7] at 78.2% vs. 72.4% is explicitly *not* a controlled experiment at full scale, as the authors themselves note.

Limitations of SRC-12. The checklist is a judgment aid, not a guarantee; several items (e.g., item 7, the paraphrase test) require human reading, so full automation is out of reach with current tools. It has not yet been validated in a user study; measuring inter-rater agreement and time cost of SRC-12 audits is the natural next step.

Falsifiability. The framework’s central claim—reduction, not generation or verification, is the bottleneck—would be undermined by evidence of the following kind: (i) systems achieving informal-language coverage *and* formal guarantees *without* an explicit reduction layer (e.g., end-to-end NL-to-proof models whose failures are dominated by search, not translation); or (ii) demonstrations that learned verifiers match formal verification on trace-level error without any typed intermediate representation. Neither pattern is present in the literature we surveyed, but both are concrete and testable.

Ethical and societal note. Over-trusting fluent but unverified mathematical output is already a practical hazard in education and quantitative work [4]. We release SRC-12 in the hope that even its informal use raises the cost of unverified conclusions.

8 Conclusion

We reorganized the reliability landscape of AI mathematics around a three-layer model whose central component—semantic reduction from natural language to typed, verifiable objects—explains the observed gap between answer accuracy and reasoning validity. A taxonomy of five reduction failure modes, a cross-paper comparison of four verification architectures, and a 12-item operational checklist together turn this position into a usable framework. Reliable AI reasoning, on this account, is not *generation plus a final check* but *generation plus semantic reduction plus verification*—and the middle term, long implicit in every serious system, deserves to be a first-class design object.

References

- [1] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021.

- [2] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.
- [3] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.
- [4] Simon Frieder, Luca Pinchetti, Alexis Chevalier, Ryan-Rhys Griffiths, Tommaso Salvatori, Thomas Lukasiewicz, Philipp Christian Petersen, and Julius Berner. Mathematical capabilities of ChatGPT. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. arXiv:2301.13867.
- [5] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. In *NeurIPS 2022 Workshop on MATH-AI*, 2022. arXiv:2211.14275.
- [6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [7] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [8] Jin Peng Zhou, Charles E. Staats, Wenda Li, Christian Szegedy, Kilian Q. Weinberger, and Yuhuai Wu. Don’t trust: Verify—grounding LLM quantitative reasoning with auto-formalization. *arXiv preprint arXiv:2403.18120*, 2024.
- [9] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12673 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [10] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, Hening He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. doi: 10.1038/s41586-023-06747-5.
- [11] Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklos Z. Horváth, Goran Zuzic, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651: 607–613, 2025. doi: 10.1038/s41586-025-09833-y.
- [12] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [13] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research (TMLR)*, 2023.

- [14] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- [15] Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Timothée Lacroix, Jiacheng Liu, Wenda Li, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- [16] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyi Song, Sainath Yu, Saad Godil, Ryan J. Prenger, and Animashree Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.
- [17] Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco D. Santos, Stephen McAleer, Albert Q. Jiang, Jason Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [18] Thomas Hales, Mark Adams, Gertrud Bauer, Tat Dat Dang, John Harrison, Truong Le Hoang, Cezary Kaliszyk, Victor Magron, Sean McLaughlin, Tat Thang Nguyen, et al. A formal proof of the Kepler conjecture. *Forum of Mathematics, Pi*, 5:e12, 2017.
- [19] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.
- [20] Guillaume Lample, Timothée Lacroix, Marie-Anne Lachaux, Thibaut Rodriguez, Amaury Hayat, Théo Lavril, Gabriel Ebner, and Xavier Martinet. Hypertree proof search for neural theorem proving. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Table 2: SRC-12: a reliability checklist for AI-generated mathematical conclusions.

#	Layer	Check	Guards against
1	Gen	Is the final answer consistent across k independent samples?	sampling noise
2	Gen	Does the answer survive a re-derivation with a different prompt strategy?	prompt-specific shortcuts
3	Gen	Is every quantity, symbol, and assumption in the problem reflected in the solution?	F1 ambiguity
4	Red	Has arithmetic been delegated to a calculator/program rather than the LLM?	computation errors [12, 13]
5	Red	Can each step be pointed to a formal or executable counterpart?	F3 non-invertibility
6	Red	Are types and domains explicit (integers vs. reals, sets vs. tuples)?	F2 type incompleteness
7	Red	Is the formalized statement checked against the original wording (paraphrase test)?	F4 vacuous formalization
8	Ver	Was a symbolic tool run, and did it pass on the <i>reduced</i> object?	unverified claims
9	Ver	Is the checker itself versioned and reproducible (library, solver version)?	F5 format drift [16]
10	Ver	Does the guarantee cover the claim actually made (not a weaker variant)?	overclaiming
11	Both	Is there a traceable rendering of the verified object back into language?	F3 non-invertibility
12	Both	Is confidence reported as a property of the <i>process</i> , not the answer?	outcome-only bias [5]